Day2 演習6

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

def ozbudak_model( t , init , *param ):
    x = init[ 0 ]
    y = init[ 1 ]

    [ alpha , beta , rho , x_out ] = param

    dxdt = -x + alpha * x_out + beta * y
    dydt = (x ** 2) / (rho + (x ** 2)) - y

    return [ dxdt , dydt ]


init = [ 3 , 0 ]
t_span = [ 0 , 30 ]

alpha = 0.1
beta = 10
rho = 25

for i, x_out in enumerate( [ 3 , 10 ] ): # without / with hysteresis
    param = [ alpha , beta , rho , x_out ]

    solution = solve_ivp( ozbudak_model, t_span, init , method='RK45', args=param )

    TMG = solution.y[0]
    LacY = solution.y[1]

    plt.subplot( 1 , 2 , i + 1 )
    plt.title( f'x_out = {x_out}' )
    plt.xlabel( 'x (TMG)' )
    plt.ylabel( 'y (LacY-GFP)' )

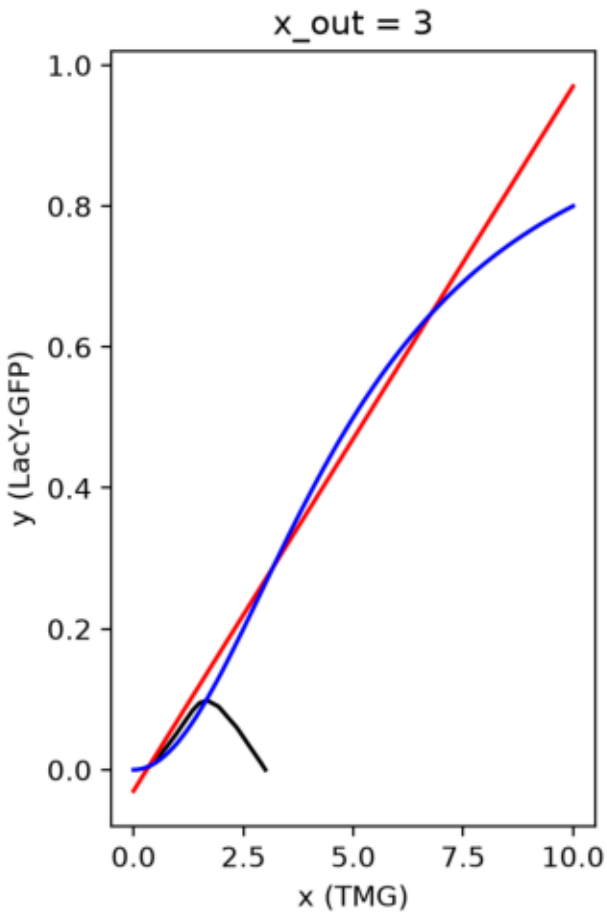
    plt.plot(TMG, LacY, 'k-')

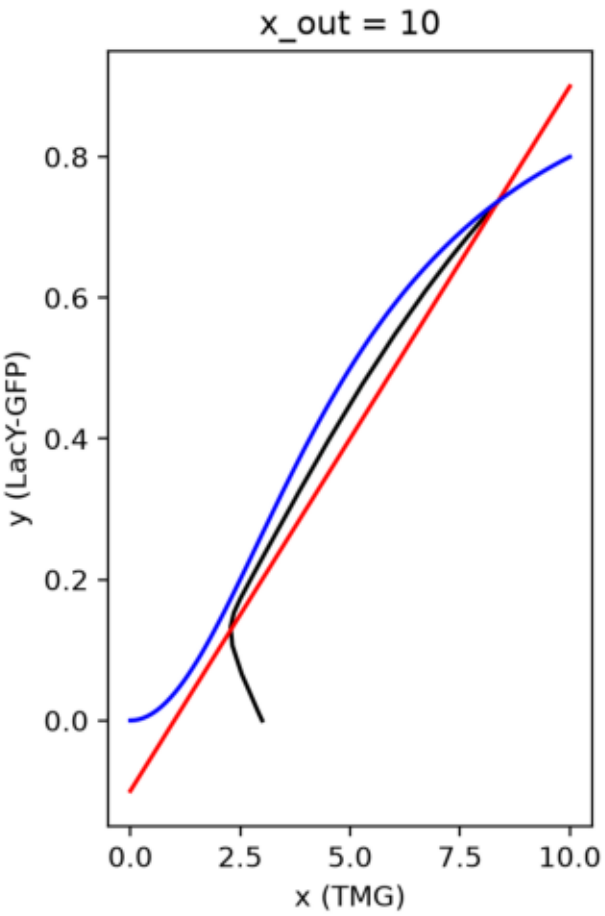
    x = np.linspace( 0, 10, 100 )

    x_null = ( x - alpha * x_out ) / beta # The nullcline for dxdt=0
    y_null = x ** 2 / (rho + x ** 2) # The nullcline for dydt=0

    plt.plot( x , x_null , 'r-')
    plt.plot( x , y_null , 'b-')

plt.tight_layout()
plt.show()
```


x_out = 3
y (LacY-GFP)
x (TMG)
0.0
0.2
0.4
0.6
0.8
1.0
0.0
2.5
5.0
7.5
10.0


x_out = 10
y (LacY-GFP)
x (TMG)
0.0
0.2
0.4
0.6
0.8
0.0
2.5
5.0
7.5
10.0

Day3 演習1

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

def selkov_model( t , init , *param ):
    x = init[ 0 ]
    y = init[ 1 ]

    [ a , b ] = param


    v1 = b
    v2 = a * y + (x ** 2) * y
    v3 = x

    dxdt = v2 - v3
    dydt = v1 - v2

    return [ dxdt , dydt ]


init = [ 1.0 , 1.0 ]
t_span = [ 0 , 100 ]
t_eval = np.linspace( t_span[0] , t_span[1] , 10000 )

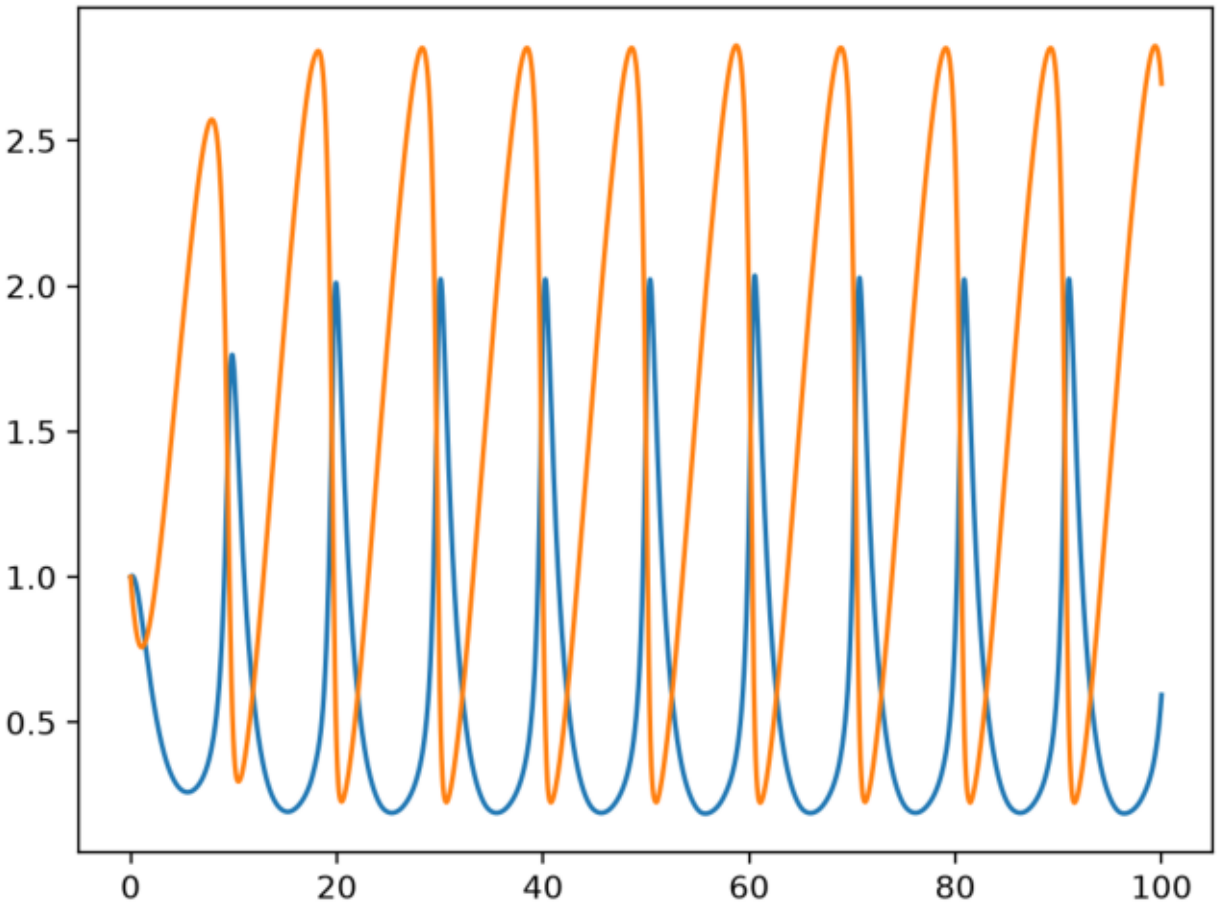
a = 0.06
b = 0.6

param = [ a , b ]

solution = solve_ivp( selkov_model, t_span, init, t_eval=t_eval, method='RK45',
                      args=param )

ADP = solution.y[0]
F6P = solution.y[1]

plt.plot( solution.t , ADP )
plt.plot( solution.t , F6P )
plt.show()
```


2.5
2.0
1.5
1.0
0.5
0
20
40
60
80
100

Day3 演習2

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

def selkov_model( t , init , *param ):
    x = init[ 0 ]
    y = init[ 1 ]

    [ a , b ] = param


    v1 = b
    v2 = a * y + (x ** 2) * y
    v3 = x

    dxdt = v2 - v3
    dydt = v1 - v2

    return [ dxdt , dydt ]


init = [ 1.0 , 1.0 ]
t_span = [ 0 , 100 ]
t_eval = np.linspace( t_span[0] , t_span[1] , 10000 )

a = 0.06
b = 0.6

param = [ a , b ]

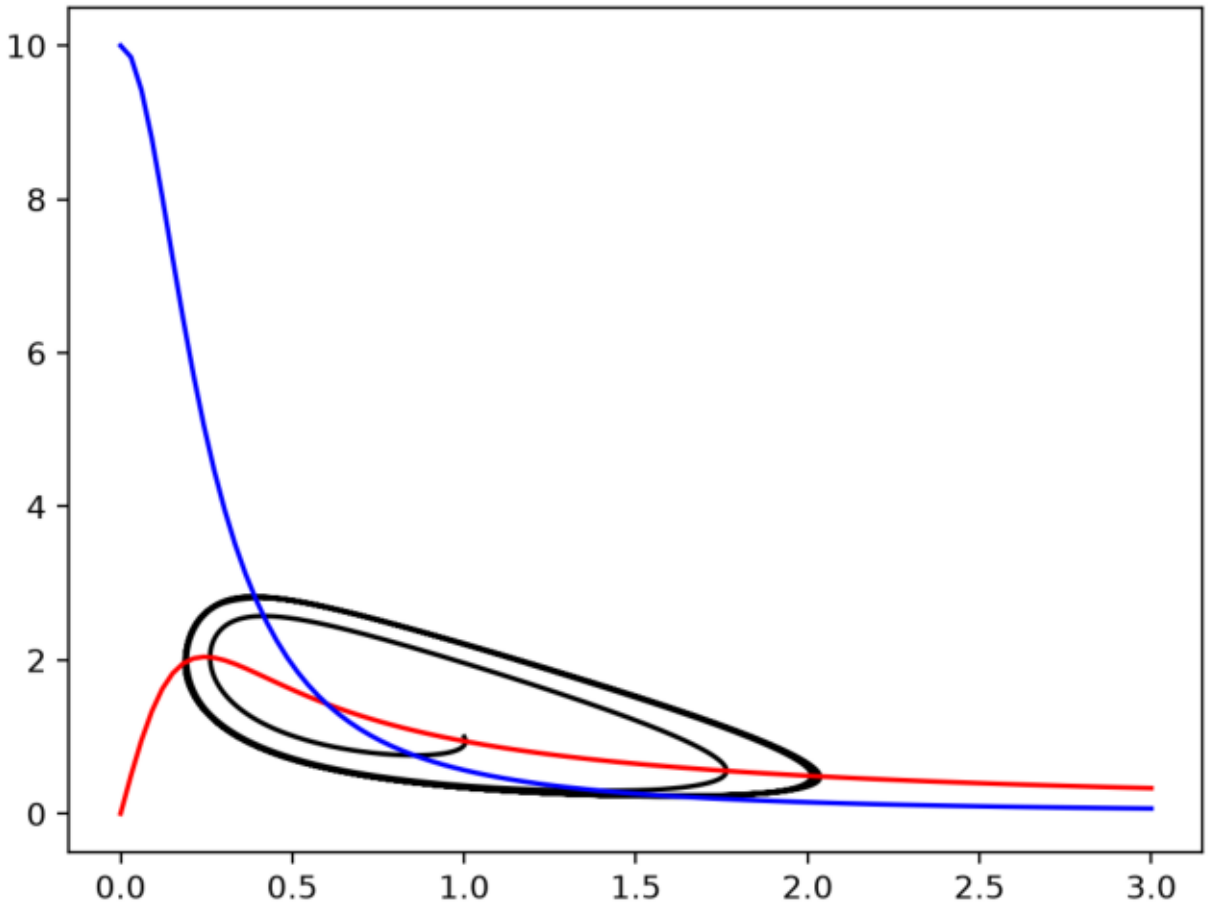
solution = solve_ivp( selkov_model, t_span, init, t_eval=t_eval, method='RK45',
                      args=param )

ADP = solution.y[0]
F6P = solution.y[1]

plt.plot(ADP, F6P, 'k-')

x = np.linspace( 0, 3, 100 )
x_null = x / ( a + x ** 2 ) # The nullcline for dydt
y_null = b / ( a + x ** 2) # The nullcline for dydt

plt.plot( x, x_null, 'r-')
plt.plot( x, y_null, 'b-')
plt.show()
```


10
8
6
4
2
0
0.0
0.5
1.0
1.5
2.0
2.5
3.0

Day3 演習3

1. ベクトル場

```
import matplotlib.pyplot as plt
import numpy as np

N = 10
x = np.linspace( 0, 3, N )
y = np.linspace( 0, 10, N )

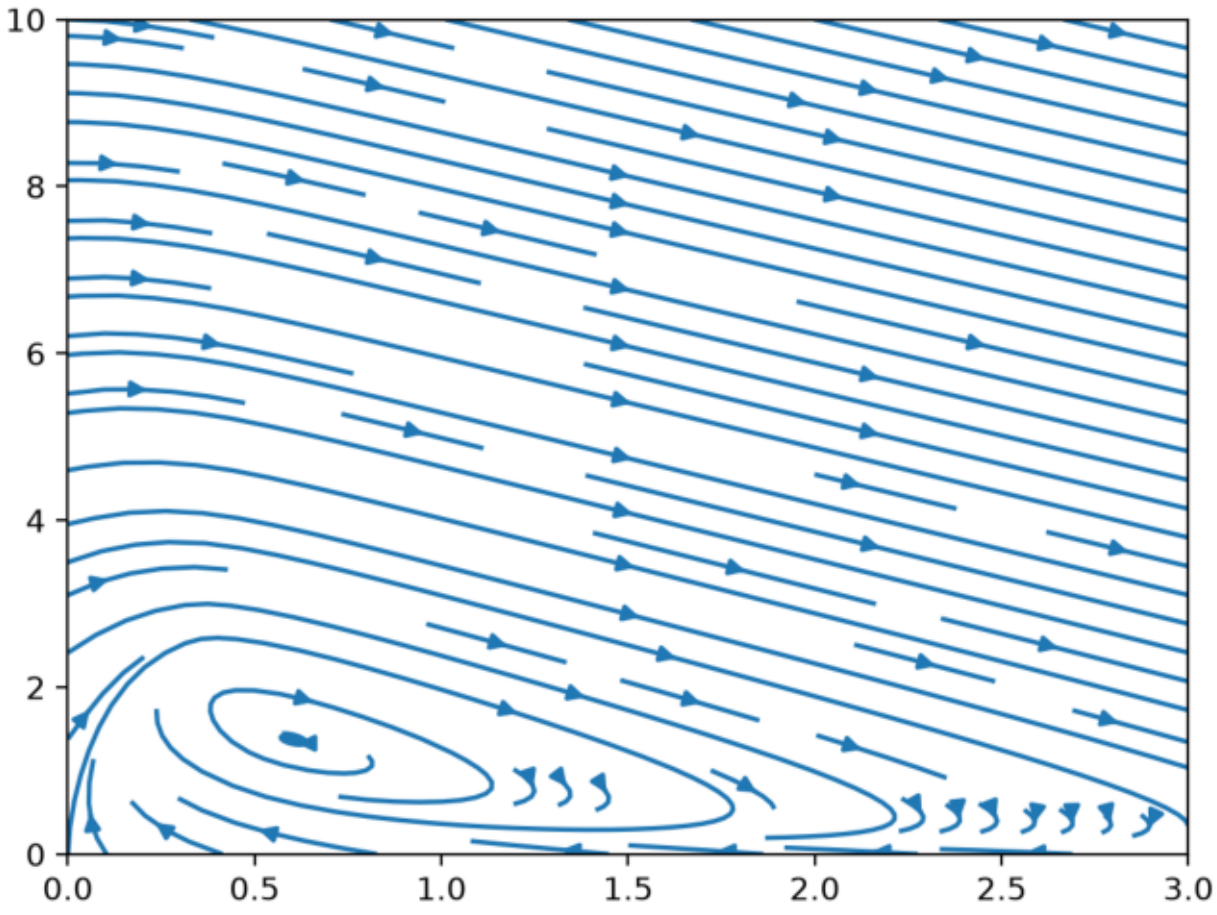
[X, Y] = np.meshgrid( x, y )

a = 0.06
b = 0.6

v1 = b
v2 = a * Y + (X ** 2) * Y
v3 = X

dxdt = v2 - v3
dydt = v1 - v2

# plt.quiver( X, Y, dxdt, dydt)
plt.streamplot( X, Y, dxdt, dydt)
plt.show()
```



2. ヌルクライン・解軌跡との重ね描き

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

def selkov_model( t , init , *param ):
    x = init[ 0 ]
    y = init[ 1 ]

    [ a , b ] = param


    v1 = b
    v2 = a * y + (x ** 2) * y
    v3 = x

    dxdt = v2 - v3
    dydt = v1 - v2

    return [ dxdt , dydt ]


init = [ 1.0 , 1.0 ]
t_span = [ 0 , 100 ]
t_eval = np.linspace( t_span[0] , t_span[1] , 10000 )

a = 0.06
b = 0.6

param = [ a , b ]

# Trajectory
solution = solve_ivp( selkov_model, t_span, init, t_eval=t_eval, method='RK45',
                      args=param )

ADP = solution.y[0]
F6P = solution.y[1]

# Nullclines
x = np.linspace( 0, 3, 100 )
x_null = x / ( a + x ** 2 ) # The nullcline for dxdt
y_null = b / ( a + x ** 2) # The nullcline for dydt

# Vector field (grid names xg, yg to avoid overwriting x)
N = 10
xg = np.linspace( 0, 3, N )
yg = np.linspace( 0, 10, N )

[X, Y] = np.meshgrid( xg, yg )

dXdt = ( a * Y + (X ** 2) * Y ) - X # v2 - v3
dYdt = b - ( a * Y + (X ** 2) * Y ) # v1 - v2

for i, method in enumerate( [ 'quiver' , 'streamplot' ] ):
    plt.subplot( 1 , 2 , i + 1 )
    plt.title( method )

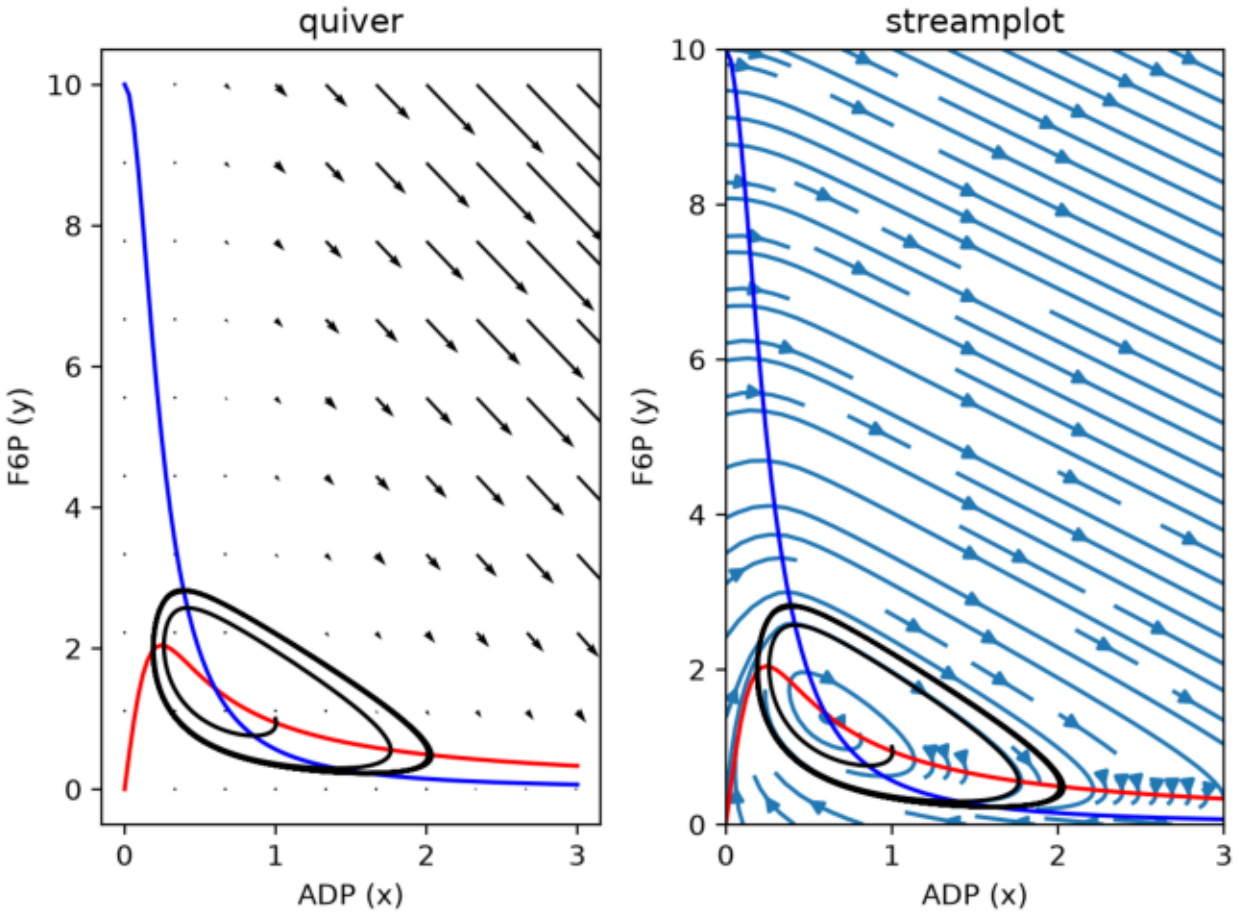
    if method == 'quiver':
        plt.quiver( X, Y, dXdt, dYdt )
```

```
    else:
        plt.streamplot( X, Y, dXdt, dYdt )

    plt.plot( x, x_null, 'r-')
    plt.plot( x, y_null, 'b-')
    plt.plot( ADP, F6P, 'k-')

    plt.xlabel( 'ADP (x)' )
    plt.ylabel( 'F6P (y)' )

plt.tight_layout()
plt.show()
```



Day3 演習4

固定点

```
import sympy as sp

# 変数と定数
x = sp.Symbol('x')
y = sp.Symbol('y')

a = 0.06
b = 0.6

# 交点の座標を求める
solutions = sp.solve( [-x + a * y + (x ** 2) * y, b - a * y - (x ** 2) * y]  , ( x ,
                      y ) )

print(solutions)
```

実行結果

```
[(0.600000000000000, 1.42857142857143)]
```

線形化・ヤコビ行列・固有値

```
import sympy as sp

x, y, a, b = sp.symbols("x y a b", real=True, positive=True)

a_val = 0.06
b_val = 0.6

f = -x + a * y + x**2 * y
g = b - a * y - (x ** 2) * y

# ヌルクラインと固定点
solutions = sp.solve( [f, g]  , ( x , y ) )
fix_x , fix_y = solutions[0]

# ヤコビ行列
J_sym = sp.Matrix( [ [ f.diff( x ) , f.diff( y ) ] , [ g.diff( x ) , g.diff( y ) ] ] )
J_num = J_sym.subs( [ ( x , fix_x ) , ( y , fix_y ) , ( a , a_val ), ( b , b_val ) ] )

print( J_sym )
print( J_num )
print( J_num.eigenvals() )
```

実行結果

```
Matrix([[2*x*y - 1, a + x**2], [-2*x*y, -a - x**2]])
Matrix([[0.714285714285714, 0.420000000000000], [-1.71428571428571,
        -0.420000000000000]])
{0.147142857142857 - 0.631148936140937*I: 1, 0.147142857142857 + 0.631148936140937*I: 1}
```

Day3 演習5

1. 固有値の虚実・正負と固定点の分類

固有値: $\lambda = 0.147 \pm 0.631i$ より、固有値は複素数であり、実部は正である。
そのため、固定点の周りを回転するスパイラルを形成し、回るたびに固定点から遠ざかる。
よって、固定点の分類は不安定スパイラルである。

2. 固定点は安定か不安定か

1より、該当する固定点は不安定である。
まず、固有値の実部は $+0.147 > 0$より、固定点からわずかにズレると、時間と共に $e^{0.147t}$ のようにズレていき、固定点に戻らない。
また、演習2・3の相平面を見ると、解の軌跡は $(0.6,\ 1.43)$ の周りを外側に広がるように回転しており、リミットサイクルに乗った。
さらに、ADPおよびF6Pは減衰せずに振動を続けているため、固定点は不安定であるとわかる。

Day3 演習6

```
import matplotlib.pyplot as plt
import numpy as np
import sympy as sp
from scipy.integrate import solve_ivp

def selkov_model( t , init , *param ):
    x = init[ 0 ]
    y = init[ 1 ]

    [ a , b ] = param


    v1 = b
    v2 = a * y + (x ** 2) * y
    v3 = x

    dxdt = v2 - v3
    dydt = v1 - v2

    return [ dxdt , dydt ]


# Jacobian (symbolic)
xs, ys, as_, bs = sp.symbols("x y a b", real=True, positive=True)

f = -xs + as_ * ys + xs**2 * ys
g = bs - as_ * ys - (xs ** 2) * ys

solutions = sp.solve( [f, g]  , ( xs , ys ) )
fix_x , fix_y = solutions[0]

J_sym = sp.Matrix( [ [ f.diff( xs ) , f.diff( ys ) ] , [ g.diff( xs ) ,
                   g.diff( ys ) ] ] )


init = [ 1.0 , 1.0 ]
t_span = [ 0 , 100 ]
t_eval = np.linspace( t_span[0] , t_span[1] , 10000 )

b = 0.6

plt.figure( figsize=( 11 , 8 ) )

for i, a in enumerate( [ 0.14 , 0.06 ] ): # condition 1 / condition 2
    # Eigenvalues at the fixed point
    J_num = J_sym.subs( [ ( xs , fix_x ) , ( ys , fix_y ) , ( as_ , a ), ( bs , b ) ] )
    print( f'a = {a}' )
    print( J_num )
    print( J_num.eigenvals() )

    # Simulation
    param = [ a , b ]
    solution = solve_ivp( selkov_model, t_span, init, t_eval=t_eval, method='RK45',
                          args=param )

    ADP = solution.y[0]
```

```
    F6P = solution.y[1]

    # Time series
    plt.subplot( 2 , 2 , i + 1 )
    plt.title( f'a = {a}: time series' )
    plt.plot( solution.t , ADP )
    plt.plot( solution.t , F6P )
    plt.xlabel( 't' )
    plt.ylabel( 'concentration' )

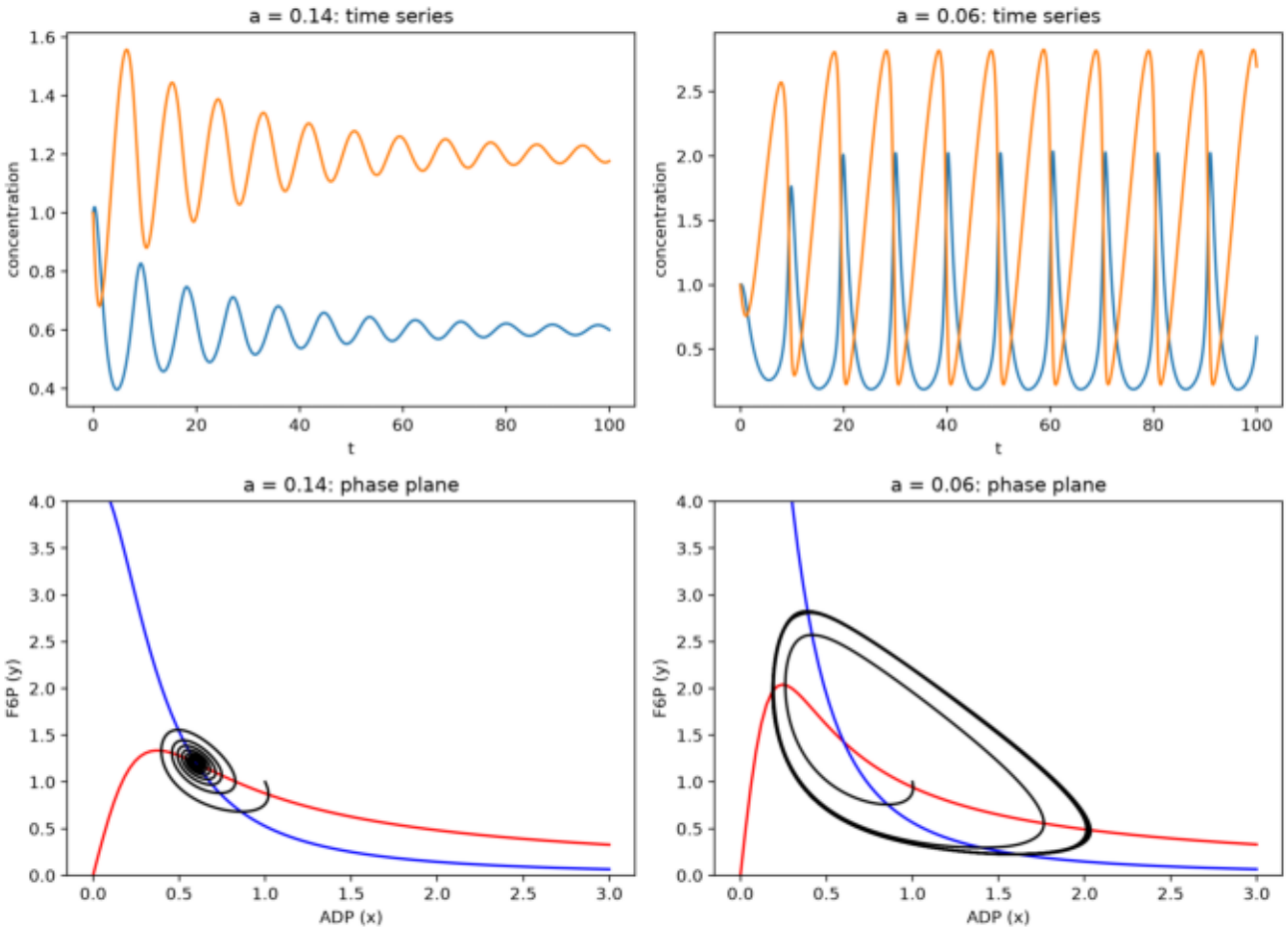
    # Phase plane
    x = np.linspace( 0, 3, 100 )
    x_null = x / ( a + x ** 2 ) # The nullcline for dxdt
    y_null = b / ( a + x ** 2) # The nullcline for dydt

    plt.subplot( 2 , 2 , i + 3 )
    plt.title( f'a = {a}: phase plane' )
    plt.plot( x, x_null, 'r-')
    plt.plot( x, y_null, 'b-')
    plt.plot( ADP, F6P, 'k-')
    plt.ylim( 0 , 4 )
    plt.xlabel( 'ADP (x)' )
    plt.ylabel( 'F6P (y)' )

plt.tight_layout()
plt.show()
```

実行結果

```
a = 0.14
Matrix([[0.440000000000000, 0.500000000000000], [-1.44000000000000,
        -0.500000000000000]])
{-0.03 - 0.706470098447203*I: 1, -0.03 + 0.706470098447203*I: 1}
a = 0.06
Matrix([[0.714285714285714, 0.420000000000000], [-1.71428571428571,
        -0.420000000000000]])
{0.147142857142857 - 0.631148936140937*I: 1, 0.147142857142857 + 0.631148936140937*I: 1}
```


a = 0.14: time series
concentration
t
a = 0.06: time series
concentration
t
a = 0.14: phase plane
F6P (y)
ADP (x)
a = 0.06: phase plane
F6P (y)
ADP (x)